

Reviewer Pack — Minimal Topological Entropy Bounds Communication Complexity

Entry bf6197d76f2b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 07:28:35 UTC

Minimal Topological Entropy Bounds Communication Complexity

Entry ID: bf6197d76f2b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 07:28:35 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics (specifically, entropy theory) **Field B** (complexity object): Communication Complexity

Statement:

For every CNF φ with m clauses and n variables, the minimal topological entropy of the associated shift dynamical system is linearly correlated with its communication complexity, such that $H_{\min}(\varphi) = \Theta(C(\varphi))$ where $C(\varphi)$ is the communication complexity of φ .

Rationale (proposer's reasoning):

Topological dynamics has been used to study entropy in various systems, but its application to communication complexity is rare. This conjecture suggests a potential link between the complexity of computations and the dynamical behavior of systems, which could reveal new insights into the nature of computation.

Taxonomy category: Topological_Dynamics (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d767d0b1f590f3cc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the CNFs show a Pearson correlation coefficient between $H_{\min}(\varphi)$ and $C(\varphi)$ greater than or equal to 0.7, with an average $H_{\min}(\varphi)$ value within 3 standard deviations of the expected linear relationship.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Topological Dynamics AND Communication Complexity - entropy theory AND shift dynamical system AND communication complexity-minimal topological entropy AND linearly correlated AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1710.08147v1] A 3D Human Body Blockage Model for Outdoor Millimeter-Wave Cellular Communication - [http://arxiv.org/abs/physics/0412058v2] Complex Dynamics of Autonomous Communication Networks and the Intelligent Communication Paradigm - [http://arxiv.org/abs/1609.02093v3] Dynamical complexity and controlled operator K-theory - [http://arxiv.org/abs/1612.05917v1] Linear Quantum Entropy and Non-Hermitian Hamiltonians - [http://arxiv.org/abs/1909.12897v1] Distance Estimation Methods for a Practical Macroscale Molecular Communication System - [http://arxiv.org/abs/1809.02144v1] Topological entropy for locally linearly compact vector spaces and field extensions - [http://arxiv.org/abs/2403.07054v3] Minimal Fractional Topological Insulator in half-filled conjugate moiré Chern bands

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def communication_complexity(cnf):
        m = len(cnf)
        n = max(abs(x) for x in sum(cnf, []))
        complexity = 0
        for clause in cnf:
            complexity += n - abs(sum(clause))
        return complexity / (m * n)

    def topological_entropy(cnf):
        m = len(cnf)
```

```

n = max(abs(x) for x in sum(cnf, []))
entropy = 0
for i in range(m):
    for j in range(n):
        if random.choice([True, False]):
            entropy += 1 / (m * n)
return entropy

results = []
for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5): # Ensure at least 30 instances per seed
        cnf = generate_cnf(n, random.randint(1, n * 2))
        C = communication_complexity(cnf)
        H_min = topological_entropy(cnf)
        results.append((C, H_min))

if not results:
    return {
        "metric_name": "H_min vs C",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

H_min_values = [H for _, H in results]
C_values = [C for C, _ in results]

mean_H_min = sum(H_min_values) / len(H_min_values)
std_H_min = math.sqrt(sum((x - mean_H_min) ** 2 for x in H_min_values) / len(H_min_values))

correlation_coefficient = sum((C - mean_C) * (H - mean_H_min) for C, H in results) / \
    (len(results) * std_C * std_H_min)

return {
    "metric_name": "H_min vs C",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for n, _ in [(5, 10, 15, 20, 30, 40)[i] for i in range(len(results))]),
    "conjecture_holds": correlation_coefficient >= 0.7 and abs(mean_H_min - (0.5 * sum(C_values))) < 0.1,
    "counterexample": "" if correlation_coefficient >= 0.7 else f"correlation={correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"metric_value\": {trial_result['metric_value']}\"}}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["metric_value"] is not None for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['conjecture_holds']}\"")
else:
    print(f"RESULT: INCONCLUSIVE reason=not_enough_data n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f52f4ea3.py", line 88, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f52f4ea3.py", line 70, in run_trial
    correlation_coefficient = sum((C - mean_C) * (H - mean_H_min) for C, H in results) / \
                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f52f4ea3.py", line 70, in <genexpr>
    correlation_coefficient = sum((C - mean_C) * (H - mean_H_min) for C, H in results) / \
                               ^^^^^^^
NameError: name 'mean_C' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a NameError, which prevented the computation of the Pearson correlation coefficient and other necessary metrics. | next: Investigate the error in the code that caused the crash and fix it before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19732
2	propose	ollama_remote	glm4:latest	0	0	18679
3	preregistration	ollama_remote	glm4:latest	0	0	14724
4	novelty	ollama_remote	glm4:latest	0	0	8246

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	12193
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15593
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12559
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10015
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13691
10	judge	ollama_remote	glm4:latest	0	0	11837

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137268 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/bf6197d76f2b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/bf6197d76f2b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/bf6197d76f2b.tar.gz* (if generated)